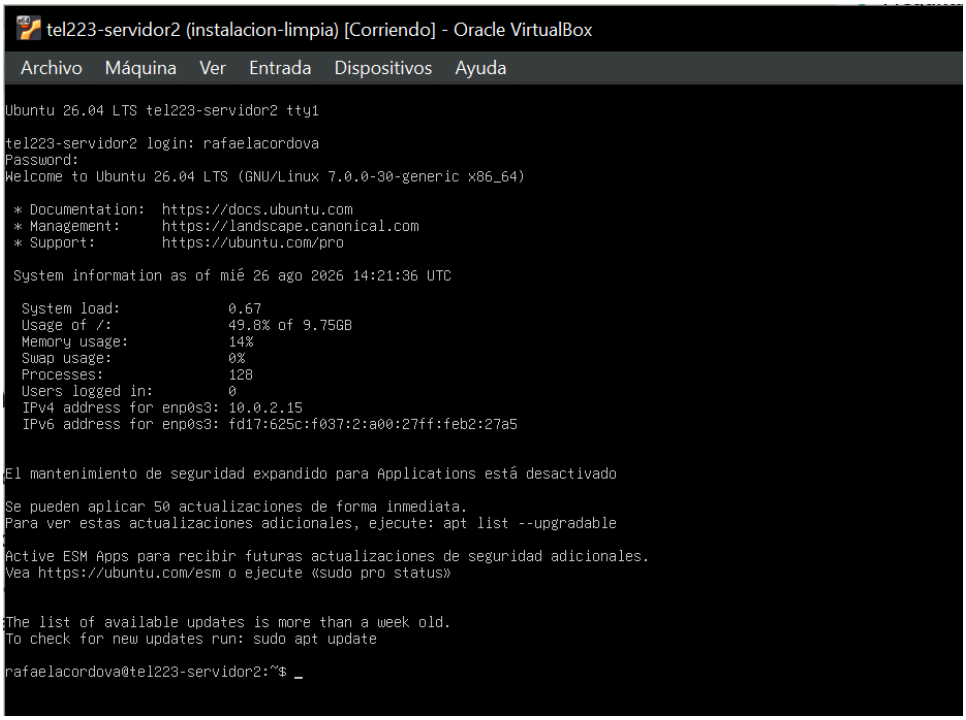
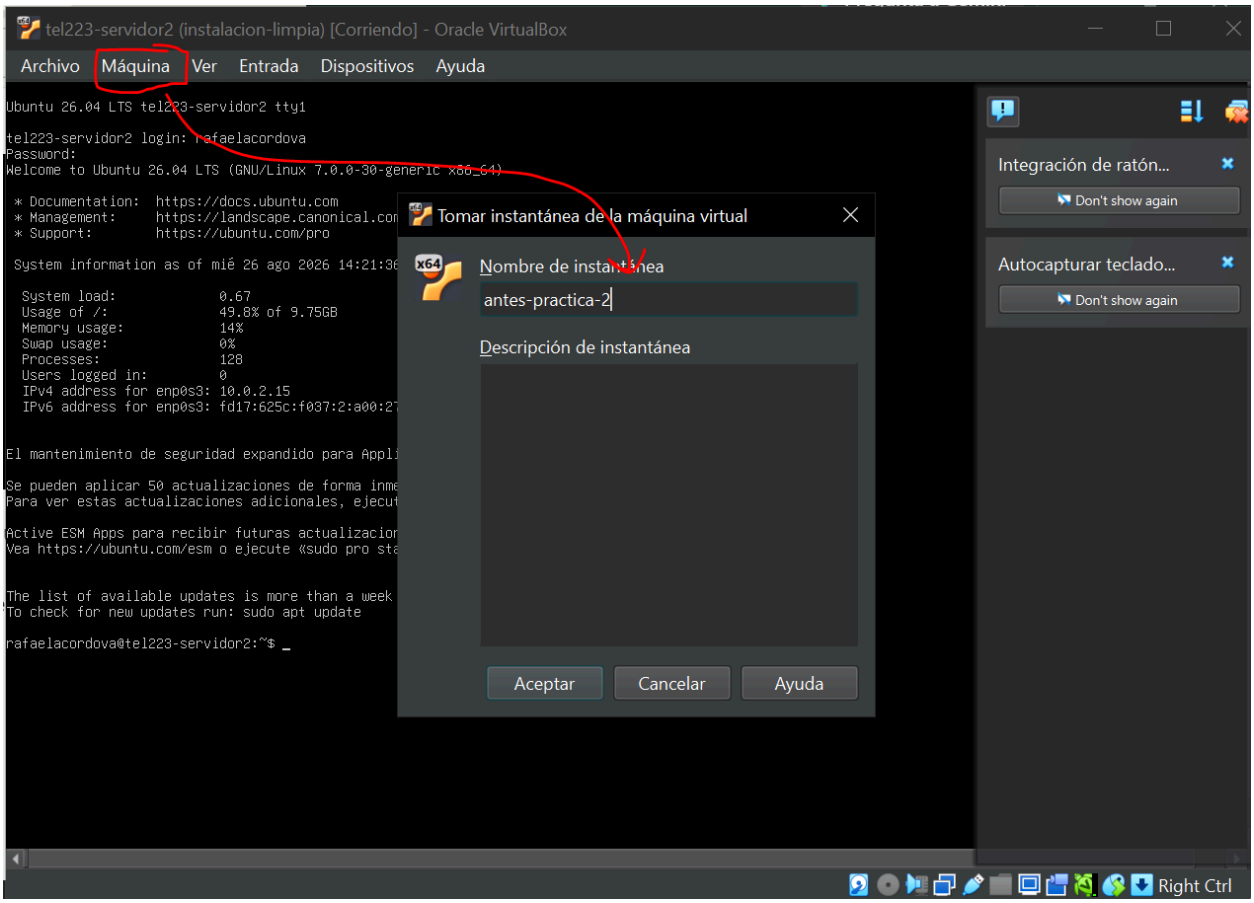
# LABORATORIO #2 - Fallas 203/EXEC en archivo .service

### Etapa 1: Crear el servicio

1. **Encendido e inicio de sesión**: Encienda su máquina virtual e ingrese con su usuario registrado.




1. **Tomar instantánea**: Antes de realizar cualquier cambio, en el menú superior de VirtualBox vaya a **Máquina > Instantáneas > Tomar** y asígnele el nombre `antes-practica-2`.




1. **Crear el script de ejecución**:Bash

   Ejecute en la terminal:

```
sudo nano /usr/local/bin/latido.sh
```

> 💡 Este comando le ordena al sistema ***abrir*** un ***editor de texto*** con ***permisos*** de ***administrador*** para ***crear*** o ***modificar*** un ***archivo ejecutable*** en una ***carpeta específica*** del sistema.
>
> - `sudo` **(*SuperUser DO* / Ejecutar como Administrador):** En Linux, por seguridad, un usuario normal no tiene permiso para modificar los archivos principales del sistema. Usar `sudo` equivale a dar clic derecho en Windows y seleccionar *"Ejecutar como Administrador"*: le otorga al comando el ***poder total del sistema*** para poder guardar el archivo.

- `nano`**:** Es un ***editor de texto básico*** que funciona directamente dentro de la pantalla negra (terminal). Es el equivalente al ***"Bloc de Notas"*** de Windows, pero sin interfaz gráfica.
- `/usr/local/bin/` **(La Ruta / Carpeta):** Es la ubicación exacta dentro del disco donde se guardará el archivo. En los sistemas Linux, la carpeta `/usr/local/bin` es el ***lugar oficial reservado para guardar programas creados por el usuario*** que el sistema debe poder ejecutar desde cualquier lugar.
- `latido.sh` **(El Archivo):** Es el nombre del archivo. La extensión `.sh` significa que es un ***script*** *de* ***Shell***, es decir, un archivo de texto que contiene ***instrucciones*** que el sistema operativo ejecutará de forma automática.

Si necesita explicarlo brevemente en sus propias palabras a un docente o compañero, la idea central es: *"Estamos **abriendo el Bloc de Notas** (`nano`) como Administrador (`sudo`) para escribir un **archivo de instrucciones** (`latido.sh`) dentro de la carpeta de programas del sistema (`/usr/local/bin/`)".*



**¿Qué significa `sh` exactamente?**

```
sh
```

viene de la palabra
**Shell**
(que es la terminal o pantalla negra de comandos). El
*Shell*
es el intérprete del sistema operativo: el programa que traduce lo que tú tecleas a un lenguaje que el procesador pueda entender. Al ponerle

```
.sh
```

a un archivo, le estás avisando al sistema:
*"este archivo contiene comandos de terminal"*
.



La clave de la automatización no está en cuántas veces escribes el archivo, sino en **cuántas miles de veces la computadora ejecutará esas órdenes por ti sin que vuelvas a tocar el teclado**.
**Ejemplo cotidiano: La alarma de tu teléfono**

- **Sin automatizar:** Todos los días a las 6:59 AM te despiertas, miras el reloj, esperas a que dé las 7:00 AM y haces sonar un timbre manual. Lo haces 365 veces al año.
- **Automatizado:** Una sola vez en la vida entras a la app del reloj, configuras *"Sonarm a las 7:00 AM de lunes a viernes"* y guardas la regla. Te costó un minuto escribir esa regla, pero la alarma trabajará sola 250 días al año.

**Ejemplo en Linux: Tareas del servidor**
Imagina que cada mañana al llegar al trabajo debes hacer 3 cosas en la terminal:

1. Borrar archivos temporales.
2. Hacer una copia de seguridad.
3. Revisar si hay conexión a Internet.

- **Camino Manual:** Cada mañana te sientas, escribes el comando 1 (Enter), esperas, escribes el comando 2 (Enter), esperas, escribes el comando 3 (Enter).
- **Camino Automatizado (`.sh`):** El primer día tardas 2 minutos en redactar un archivo `mantenimiento.sh` con esas 3 instrucciones. A partir de ahí, solo ejecutas ese archivo una vez (o dejas que el sistema lo ejecute solo al encender) y Linux procesa las 3 tareas en 1 segundo.

**El caso exacto de su práctica (`latido.sh`)**

En la guía de laboratorio se escribió este código dentro del archivo `latido.sh`:
Bash

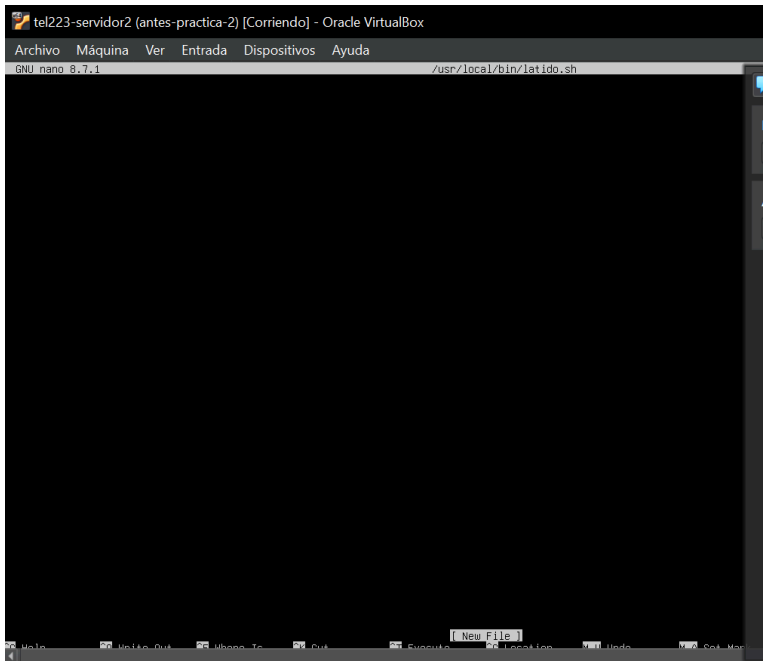
```
while true; do
    echo "latido $(date '+%H:%M:%S')"
    sleep 5
done
```

Si usted tuviera que hacer esto "a mano" sin el archivo `.sh`, tendría que mirar el reloj de su pared y escribir en la pantalla la frase `latido 10:15:00`, esperar 5 segundos en el cronómetro, y volver a escribir `latido 10:15:05`
Tendría que hacerlo **17.280 veces al día** sin parar ni para dormir.

Al escribir el archivo `.sh` una sola vez, la computadora repite esa instrucción automáticamente cada 5 segundos de forma indefinida[cite: 1].

**Cómo explicarlo en sus palabras a otra persona:**
"Escribir el archivo `.sh` requiere redactar las instrucciones una sola vez. La ventaja es que, una vez guardado, el servidor puede repetir esa lista de comandos miles de veces o dejarla corriendo en segundo plano sin que un humano tenga que volver a teclear nada."

Pide el Password antes de pasar a la siguiente pantalla de abajo




Escriba exactamente esto:

```
#!/bin/bash
while true; do
    echo "latido $(date '+%H:%M:%S')"
    sleep 5
done
```

Este pequeño programa es un **escritor automático de bitácora**: su único trabajo es despertarse cada 5 segundos, mirar la hora del reloj del sistema e imprimir un mensaje en pantalla para demostrar que el servidor sigue con "vida" (igual que un pulso cardiaco).
**Línea 1:** `#!/bin/bash` **(El "Shebang")**

- **¿Qué hace?** Le indica al sistema operativo qué programa debe usar para leer y ejecutar este archivo.
- **Explicación sencilla:** Es como la portada de un libro que dice *"Este libro está escrito en castellano"*. En Linux, `/bin/bash` es el "traductor" de comandos más común. Si olvida esta línea, la máquina no sabrá cómo interpretar el texto y el servicio fallará.

**Línea 2 y 5:** `while true; do` **...** `done` **(El bucle infinito)**

- `while true; do`: Significa *"mientras la verdad sea verdadera (es decir, siempre), haz lo siguiente..."*.
- `done`: Marca el punto donde terminan las instrucciones que deben repetirse.
- **Explicación sencilla:** Es un reproductor en modo bucle. Todo lo que esté atrapado entre la orden `do` (iniciar) y `done` (terminar) se repetirá sin parar para siempre hasta que alguien apague la máquina o detenga el servicio.

**Línea 3:** `echo "latido $(date '+%H:%M:%S')"` **(Escribir la hora)**

- `echo`: Es el comando para "imprimir" o escribir un texto en pantalla (el equivalente al mensaje de voz o texto del programa).
- `"latido ..."`: La palabra literal que queremos que escriba.
- `$(date '+%H:%M:%S')`: Le dice a Linux: *"ve al reloj interno, consulta la hora exacta en formato Horas:Minutos:Segundos y pégala aquí"*.
- **Resultado:** Imprime en el registro algo como: `latido 14:30:05`.

**Línea 4:** `sleep 5` **(La pausa)**

- **¿Qué hace?** Congela la ejecución del script durante 5 segundos antes de volver al inicio del bucle.
- **Explicación sencilla:** Sin esta línea, la computadora ejecutaría el bucle millones de veces por segundo, llenando el disco de texto y congelando el servidor. El comando `sleep 5` le dice: *"descansa 5 segundos y vuelve a tomarte el pulso"*.

**Cómo explicar todo el script en sus palabras (en un solo párrafo):**

> "Es una orden en bucle infinito (`while true`). En cada vuelta escribe la palabra 'latido' junto con la hora exacta del reloj (`echo` y `date`), espera 5 segundos (`sleep 5`) y vuelve a empezar. Todo se ejecuta bajo el lenguaje Bash (`#!/bin/bash`)."

💡 Para entender `#!/bin/bash`, piense en cómo abrimos archivos en una computadora normal:
En Windows, cuando usted hace doble clic en un archivo llamado `tarea.docx`, la computadora mira la terminación `.docx` y sabe automáticamente que debe abrirlo con Microsoft Word.
En Linux, los scripts de terminal no dependen de la extensión del nombre. En su lugar, Linux abre el archivo, **lee la mismísima primera línea** y encuentra la instrucción de qué programa debe usar para procesarlo.

## Desglose parte por parte

### 1. Los dos primeros símbolos: `#!` (El "Shebang")

- **Nombre técnico:** Se le conoce popularmente como **Shebang** (combinación de *Hash* `#` y *Bang* `!`).
- **¿Qué significa para la computadora?** Es una señal de socorro o una llamada de atención para el sistema operativo. Le dice a Linux:

> *"¡Atención! Lo que viene a continuación no es una orden directa de la computadora; es un texto que debe ser leído por un programa traductor especifico."*

### 2. La ruta: `/bin/bash`

- **¿Qué es?** Es la dirección exactísima en el disco duro donde vive el programa **Bash** (el intérprete o traductor de comandos de Linux).
- `/bin/`: Es la carpeta del sistema donde guardan los programas esenciales (los binarios).
- `bash`: Es el nombre del programa ejecutable (*Bourne Again SHell*), el lenguaje en el que escribimos los comandos.

## La analogía perfecta

Imagine que le entrega una hoja de papel a un mensajero que solo habla español, pero la hoja contiene una partitura musical.

- **Sin la primera línea:** El mensajero intentará leer la partitura en voz alta como si fuera texto en español, se confundirá y tirará la hoja a la basura.
- **Con `#!/bin/bash` en la cabecera:** La hoja empieza diciendo: *"Entregue este documento directamente al profesor de música en la oficina /bin/bash"*. El mensajero no intenta entender el contenido; simplemente se lo lleva al especialista correcto.

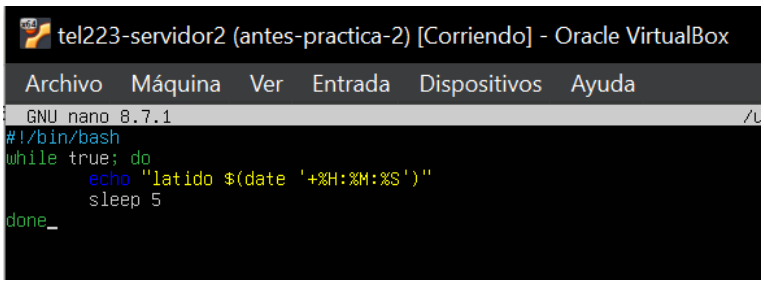
## ¿Qué pasa si olvida poner esa línea?

En la guía de laboratorio se indica explícitamente: **"Si la olvida, el servicio no arrancará"**.
Esto ocurre porque cuando el servicio de systemd intenta iniciar `latido.sh`, no sabrá qué traductor usar para procesar las órdenes `while`, `echo` o `sleep`, provocando un fallo en el arranque del servicio.

## Cómo explicarlo en sus propias palabras

Si en una evaluación o examen le preguntan qué es `#!/bin/bash`, puede responder:

> "Es la cabecera obligatoria (llamada Shebang) que le indica al sistema operativo en qué ruta exacta del disco (`/bin/bash`) se encuentra el programa traductor que debe leer y ejecutar las líneas de código de nuestro archivo."




Guarde los cambios presionando `Control + O`,




confirme con `Enter` y

salga con `Control + X`.

Control + C

**4.** Dele permiso de ejecución y compruébelo:

```
sudo chmod +x /usr/local/bin/latido.sh
ls -l /usr/local/bin/latido.sh
```

**Qué debe ver:** en la primera columna deben aparecer las x del permiso de ejecución. Si no están, el sistema no podrá ejecutarlo.



Este comando le otorga al archivo `latido.sh` la propiedad de ser reconocido como un programa ejecutable, usando privilegios de administrador para modificar sus permisos de seguridad.

1. `sudo` **(*SuperUser DO* / Ejecutar como administrador)**
   - **¿Qué hace?** Eleva los privilegios de la orden para que el sistema le permita modificar archivos protegidos del sistema.
   - **Explicación sencilla:** Sin `sudo`, Linux rechazaría el cambio porque la carpeta `/usr/local/bin/` es de nivel de sistema y los usuarios normales no pueden alterar los permisos de los archivos que residen allí.
2. `chmod` **(*CHange MODe* / Cambiar modo de acceso)**
   - **¿Qué hace?** Es el comando nativo de Linux encargado de alterar las reglas de seguridad, lectura, escritura o ejecución de un archivo o carpeta.
   - **Explicación sencilla:** Es la herramienta con la que le dices al sistema operativo: *"Quiero cambiar lo que se permite hacer con este archivo"*.
3. `+x` **(*Plus eXecute* / Añadir permiso de ejecución)**
   - `+` **(Añadir):** Significa agregar un permiso nuevo sin borrar los permisos existentes.

- `x` **(eXecute):** Es la letra asignada al permiso de **Ejecución**.
- **Explicación sencilla:** Por defecto, Linux considera que cualquier archivo nuevo creado con un editor de texto (`nano`) es solo un papel para leer o escribir. Al poner `+x`, le colocas un "sello" al archivo que le aclara a Linux: *"Este archivo no es solo texto, es un programa funcional que tienes permiso de arrancar"*.

**4.** `/usr/local/bin/latido.sh` **(La ruta objetivo)**

- **¿Qué hace?** Es la dirección exacta en el disco duro donde se ubica el archivo al que queremos aplicarle los cambios de permiso.

---

**¿Por qué es indispensable este paso?**
Si intenta iniciar un servicio sin haber ejecutado antes `chmod +x`, Linux detendrá el proceso de inmediato arrojando un error de tipo **Permission denied** (Permiso denegado), debido a que el sistema no tiene autorización explícita para correr ese texto como programa.

---

**Cómo explicarlo en sus propias palabras:**

> *"El comando usa permisos de administrador (`sudo`) para ejecutar la orden de cambio de modo (`chmod`), agregando el permiso de ejecución (`+x`) al archivo `latido.sh` (`/usr/local/bin/`), permitiendo así que el sistema operativo pueda correrlo como un programa real y no solo leerlo como un texto"*.

El segundo comando: ls -l ...



Este comando consulta y muestra en pantalla los detalles técnicos de seguridad y estado de un archivo específico, permitiendo verificar si tiene activados sus permisos de ejecución.

- `ls` **(*LiSt* / Listar):** Es el comando básico de Linux para mostrar o consultar archivos y carpetas. Equivale a abrir una carpeta en el Explorador de Windows para ver su contenido.

- `l` **(*Long* / Formato Largo o Detallado):** Es una opción (*flag*) que modifica el comportamiento de `ls`. En lugar de mostrar solo el nombre del archivo, le ordena a Linux mostrar una ficha técnica completa: permisos de seguridad, propietario, grupo, tamaño y fecha de modificación.
- `/usr/local/bin/latido.sh` **(La ruta del archivo):** Especifica la ubicación exacta del archivo sobre el cual queremos pedir el informe detallado.

**¿Qué muestra la pantalla al ejecutarlo y por qué importa?**
Al escribir este comando, la terminal devuelve una línea similar a esta:

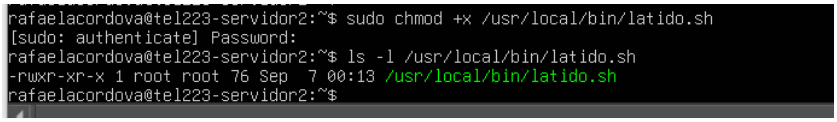
- `rwxr-xr-x 1 root root 85 Aug 24 10:00 /usr/local/bin/latido.sh`

La parte crucial son las **letras de la primera columna** (`-rwxr-xr-x`):

- `r` **(*read*):** Permiso de lectura.
- `w` **(*write*):** Permiso de escritura o modificación.
- `x` **(*eXecute*):** Permiso de ejecución.

La presencia explícita de la letra `x` confirma que el comando anterior (`sudo chmod +x`) funcionó correctamente y que el sistema operativo ya reconoce al archivo como un programa ejecutable. Si en lugar de la `x` apareciera un guion (`-`), el servicio fallaría al intentar arrancar por falta de permisos.
**Cómo explicarlo en sus propias palabras:**

> "El comando `ls -l` le pide al sistema listar (`ls`) la ficha técnica detallada (`-l`) del archivo `/usr/local/bin/latido.sh` para verificar que la letra 'x' esté presente en sus permisos, confirmando así que el archivo ya es ejecutable".



| Componente | Valor en pantalla | Significado |
| --- | --- | --- |
| Tipo de archivo | `-` | El primer guion indica que es un archivo regular (no una carpeta ni un acceso directo). |
| Permisos de seguridad | `rwxr-xr-x` PDF | Muestra qué acciones están permitidas para cada nivel de usuario. |
| Enlaces | `1` | Número de enlaces físicos (*hard links*) apuntando a este archivo. |
| Propietario y Grupo | `root root` | El primer `root` es el usuario dueño; el segundo `root` es el grupo del sistema al que pertenece. |
| Tamaño | `76` | El peso del archivo expresado en bytes (76 bytes de código guardado). |
| Fecha de modificación | `Sep 7 00:13` | Día y hora en que se guardó el archivo por última vez. |
| Ruta del archivo | `/usr/local/bin/latido.sh` PDF | Ubicación absoluta del script en el disco duro. |



El número de enlaces físicos indica cuántos nombres de archivo distintos están apuntando al mismo contenido en el disco duro, mientras que la fecha de modificación "futura" ocurre porque la máquina virtual utiliza la zona horaria universal (UTC).

**1. Los Enlaces Físicos (*Hard Links*) explicados desde cero**

En Linux, el contenido de un archivo (los datos reales guardados en el disco) y el nombre del archivo están separados.

- **La analogía:** Piense en los datos reales del código `latido.sh` como una **casa física**. El nombre `/usr/local/bin/latido.sh` es simplemente el **letrero con la dirección** que le dice a la computadora dónde está la casa.
- **¿Qué significa el número `1` en su resultado?** Significa que actualmente existe **un solo letrero** en todo el sistema apuntando a esos datos.
- **¿Qué es un *Hard Link*?** Si usted creara un "enlace físico" (un segundo letrero con otro nombre en otra carpeta, pero apuntando exactamente a la misma casa), el número cambiaría a `2`. Lo interesante de los enlaces físicos en Linux es que si borra un letrero, la casa no se destruye; el archivo solo se borra del disco duro cuando el número de enlaces llega a `0`.

**2. Por qué la fecha de modificación marca 7 de septiembre (`Sep 7 00:13`)**

Si su reloj local marca **6 de septiembre a las 20:52** (zona horaria de Bolivia, que es **UTC-4**), la diferencia de fecha en la máquina virtual se debe a dos factores:

- **Zona Horaria en UTC (Tiempo Universal Coordinado):** Por defecto, la mayoría de los servidores Linux se instalan con la zona horaria **UTC**, la cual no se ajusta a la hora local de ningún país. Como Bolivia está 4 horas por detrás de UTC (UTC-4), al sumar 4 horas a las **20:52 PM del 6 de septiembre**, el reloj de Linux pasa automáticamente al día siguiente: las **00:52 AM del 7 de septiembre**.
- **Desfase de reloj en VirtualBox (*Time Drift*):** Si la máquina estuvo pausada, suspendida o si no sincronizó la hora exacta al encender, el reloj de la máquina virtual puede presentar unos minutos de descompensación con respecto al reloj de la computadora anfitriona.

**5.** Ahora cree el archivo de unidad, que es el servicio propiamente dicho:

```
sudo nano /etc/systemd/system/tel223-latido.service
```

Escriba exactamente esto, reemplazando el usuario por el suyo:

```
[Unit]
Description=Servicio de latido TEL-223
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/latido.sh
User=SU_USUARIO

[Install]
WantedBy=multi-user.target
```

| Línea | Qué significa |
|---|---|
| Description | El nombre que verá quien administre este servidor |
| After | Espera a que la red esté lista antes de arrancar |
| ExecStart | La ruta del programa. Es la línea que más falla |
| User | El usuario con el que corre. Nunca el administrador |
| WantedBy | Cuándo arranca solo: al llegar el sistema a su estado normal |
| **Diferencia importante:** el archivo de unidad NO necesita permiso de ejecución: el sistema solo lo lee. El script sí, porque se ejecuta. | |



Este comando abre el editor de texto con privilegios de administrador para crear el archivo de configuración que le dirá a Linux cómo lanzar y supervisar su nuevo servicio.

- `sudo` **(*SuperUser DO* / Administrador):** Otorga el poder absoluto del sistema operativo para esa orden. La carpeta donde guardaremos el

servicio está protegida y Linux no le permitirá crear archivos ahí a menos que use `sudo`.

- `nano` **(El editor de texto):** Es el programa equivalente al *Bloc de Notas* de Windows, diseñado para funcionar dentro de la terminal negra.
- `/etc/` **(Carpeta de configuraciones):** Es la carpeta del sistema donde Linux almacena las configuraciones de todos los programas.
- `systemd/system/` **(Carpeta de servicios):** Es el directorio reservado para **systemd** (el administrador maestro de programas en Linux). Toda la información sobre qué programas deben arrancar automáticamente o ejecutarse en segundo plano reside aquí.
- `tel223-latido.service` **(Nombre del archivo de unidad):** Es el nombre que le asignamos a nuestro servicio. La extensión `.service` le indica explícitamente a systemd que este archivo no es un texto cualquiera, sino una "unidad de servicio" con instrucciones de arranque.

**Resumen sencillo para explicar a otra persona:**

> "Usamos permisos de administrador (`sudo`) para abrir el Bloc de Notas (`nano`) y crear un archivo de servicio (`tel223-latido.service`) dentro de la carpeta de configuraciones de servicios del sistema (`/etc/systemd/system/`), de modo que el sistema sepa cómo ejecutarlo en segundo plano".



**La extensión `.service`**
En Linux, ponerle `.service` al final de un archivo no es un adorno: es la etiqueta que le indica a `systemd` que ese archivo es un **archivo de unidad de servicio**.
Un archivo `.service` no contiene el código de su programa (no contiene las líneas de `while` ni de `sleep`). Es solo una ficha de contrato que le indica a Linux tres cosas:

1. `ExecStart`: ¿Dónde está guardado el programa que debe ejecutar? (`/usr/local/bin/latido.sh`).

2. `User`: ¿A nombre de qué usuario se va a ejecutar?.
3. `WantedBy`: ¿En qué momento debe arrancar solo? (por ejemplo, al encender el servidor).

**La diferencia clave entre `.sh` y `.service`**

• **El archivo `.sh` es el TRABAJADOR:** Es el script que realiza la tarea concreta (en su caso, escribir *"latido"* y la hora cada 5 segundos).

• **El archivo `.service` es el JEFE / SUPERVISOR:** Es la regla en `systemd` que decide cuándo contratar al trabajador, qué permisos darle y qué hacer si el programa llega a fallar.



Cada barra diagonal `/` en Linux representa un nivel de profundidad dentro del disco duro, guiándonos desde el origen principal hasta el cajón exacto donde se administran los servicios.

- `/` **(La Raíz / Root):** Es el origen absoluto del sistema de archivos en Linux (el equivalente exacto al disco `C:\` en Windows). No existe nada en la computadora que no esté dentro de esta barra inicial.
- `etc/` **(Archivos de Configuración):** Es la carpeta donde se guardan las reglas de comportamiento y ***ajustes de todo el servidor.*** El nombre proviene del latín *et cetera* (y lo demás), porque allí reside cualquier texto que configure un programa del sistema.
- `systemd/` **(Módulo del Administrador Maestro):** Es la subcarpeta asignada al programa `systemd`, el ***software que toma el control del servidor apenas enciende para levantar los servicios*** del sistema.
- `system/` **(Servicios Locales del Servidor):** Es el directorio reservado para que el administrador cree sus propios archivos de servicio a medida. Todo archivo colocado aquí (como `tel223-latido.service`) le indica a `systemd` que debe vigilar ese programa en segundo plano.

**La analogía de la ruta física:**

Imagine la ruta `/etc/systemd/system/` como la dirección para enviar un memorándum en una empresa:

1. `/` = El edificio completo de la empresa.
2. `etc/` = El departamento de Archivo General y Reglamentos.

3. `systemd/` = La oficina del Jefe de Operaciones.
4. `system/` = La bandeja de entrada con la etiqueta *"Instrucciones prioritarias creadas por el Administrador"*.

Al guardar su archivo en esa ubicación exacta, le está dejando el memorándum de su servicio en la bandeja de entrada directa del Jefe de Operaciones de Linux.



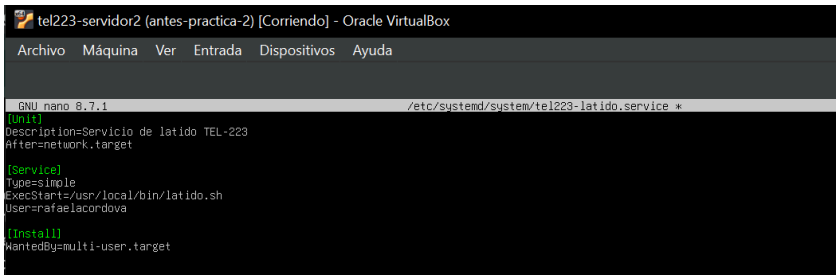
**¿A qué "servidor" nos referimos?**
Cuando se menciona *"todo el servidor"*, nos referimos a **tu propia máquina virtual (`tel223-servidor2`)** en la que estás trabajando dentro de VirtualBox.
En informática, un "servidor" no es necesariamente un edificio lleno de supercomputadoras. Un servidor es simplemente **cualquier computadora (física o virtual) configurada para ejecutar tareas o programas automáticos en segundo plano**. En esta práctica, tu máquina virtual está actuando como un servidor que va a ejecutar el servicio `tel223-latido` de forma continua.
**¿Qué es `systemd` desde cero?**
`systemd` es el **Administrador Maestro** de Linux. Es el primer programa que se despierta cuando enciendes tu computadora y el último en apagarse.

- **El problema sin `systemd`:** Si ejecutas un programa normal y cierras la pantalla, el programa se apaga. Si el programa falla o se traba, se queda muerto hasta que un humano entre a reiniciarlo.
- **La solución con `systemd`:** Es como tener un **guardia o supervisor contratado las 24 horas**. Tú le entregas una hoja de instrucciones (el archivo `.service`) y `systemd` se encarga de:
  1. Arrancar el programa automáticamente cuando enciende la computadora.
  2. Vigilarlo constantemente mientras está corriendo.
  3. Reanimarlo o reiniciarlo si el programa falla o se congela.

**¿Qué es la carpeta `system` (dentro de `/etc/systemd/system/`)?**
Dentro de la ruta donde guardamos las reglas, la subcarpeta `system/` representa las **instrucciones globales de toda la máquina**.
En Linux existen dos tipos de servicios:

1. **Servicios de Usuario:** Programas que solo funcionan cuando *tú en específico* inicias sesión (como tu reproductor de música personal).
2. **Servicios del Sistema (`system`):** Programas que le pertenecen a **toda la computadora entera**, sin importar qué usuario inicie sesión.

Al colocar tu archivo dentro de `/etc/systemd/system/`, le estás diciendo al Administrador Maestro: *"Este programa le pertenece a toda la máquina. Arráncalo en cuanto encienda el equipo, sin esperar a que ningún usuario abra su cuenta"*.




Este archivo de unidad está dividido en tres grandes bloques o "secciones" marcadas entre corchetes (`[ ]`). Cada sección agrupa reglas específicas para que el sistema operativo maneje el servicio.
**1. Sección `[Unit]` (Información General y Reglas de Inicio)**
Esta sección define los datos básicos del servicio y qué cosas deben suceder en el sistema antes de poder encenderlo:

- `[Unit]`: Es el encabezado que le avisa a `systemd`: *"A continuación vienen los datos generales del servicio"*.
- `Description=Servicio de latido TEL-223`: Es una etiqueta informativa de texto libre. Es el nombre legible que verá cualquier persona cuando consulte el estado del servidor con `systemctl status`.

- `After=network.target`: Define un orden de espera. Le ordena a `systemd` que espere a que la tarjeta de red e Internet estén completamente listas (`network.target`) antes de intentar arrancar este servicio.

**2. Sección `[Service]` (Instrucciones de Ejecución)**
Es el bloque central donde se especifica cómo, qué y quién ejecuta el programa:

- `[Service]`: Encabezado que le indica a `systemd`: *"Aquí están los detalles de cómo lanzar el programa"*.
- `Type=simple`: Especifica el modo de arranque más directo de Linux. Le indica al sistema que el programa principal se ejecutará de inmediato tan pronto como se lance la orden.
- `ExecStart=/usr/local/bin/latido.sh`: Es la ruta absoluta hacia el script que debe ejecutarse. Es la orden más importante y la que más suele fallar si hay errores ortográficos o si el archivo no existe.
- `User=SU_USUARIO`: Define con qué cuenta de usuario correrá el servicio. Por normas de seguridad informática, los programas nunca deben correr como administrador (`root`) a menos que sea estrictamente necesario.

**3. Sección `[Install]` (Reglas de Automatización)**
Define qué debe hacer el sistema cuando configuramos el programa para que encienda automáticamente al arrancar la computadora (`sudo systemctl enable`):

- `[Install]`: Encabezado que le dice a `systemd`: *"Estas son las reglas para el inicio automático"*.
- `WantedBy=multi-user.target`: Le indica al servidor en qué estado del encendido debe arrancar este programa. `multi-user.target` representa el modo normal de trabajo del servidor (cuando ya cargó todos los componentes esenciales y está listo para recibir usuarios).

**Resumen sencillo para explicar a otra persona:**

> "El archivo usa la sección `[Unit]` para nombrarlo y esperar a que la red cargue; la sección `[Service]` para indicarle qué script ejecutar

> (`ExecStart`) y bajo qué usuario correrlo (`User`); y la sección `[Install]` para engancharlo al arranque automático del servidor (`WantedBy`)."



El parámetro `Type=` le enseña a `systemd` cómo arrancar el programa y qué criterio usar para confirmar que ya está encendido, mientras que `WantedBy=` le indica en qué momento del encendido debe engancharse para arrancar solo.

**¿Qué es `Type=` y para qué sirve?**

Imagine que `systemd` es un capataz. La propiedad `Type` le indica al capataz qué estrategia debe usar para lanzar a un trabajador y cómo saber si el trabajador empezó a hacer su tarea.

- `Type=simple` **(Modo Directo)**: Es el tipo por defecto. `systemd` ejecuta la orden (`/usr/local/bin/latido.sh`) e **inmediatamente** asume que el servicio está activo y funcionando (`active running`), sin esperar ninguna confirmación del programa. Es ideal para scripts sencillos que se quedan corriendo en bucle, como nuestro latido.

**Otros tipos de `Type=` para comparar:**

- `Type=oneshot` **(Ejecutar y Salir)**: Para programas que hacen un trabajo rápido y se apagan.
  - *Ejemplo:* Un script que borra la papelera de reciclaje o actualiza la hora. Se ejecuta, limpia el disco en 3 segundos, se apaga y `systemd` lo marca como terminado.
- `Type=forking` **(Crear un proceso hijo en segundo plano)**: El programa principal arranca, crea un "hijo" en segundo plano para que trabaje en silencio, y el programa principal se cierra.
  - *Ejemplo:* Servidores web tradicionales como Apache o Nginx. El proceso inicial llama a sus trabajadores y se retira.
- `Type=notify` **(Aviso explícito)**: `systemd` lanza el programa pero se queda esperando a que el propio programa le envíe un mensaje diciendo: *"¡Ya cargué todos mis archivos pesados, estoy listo!"*.

    - *Ejemplo:* Bases de datos complejas que tardan varios segundos en iniciar antes de poder recibir conexiones.

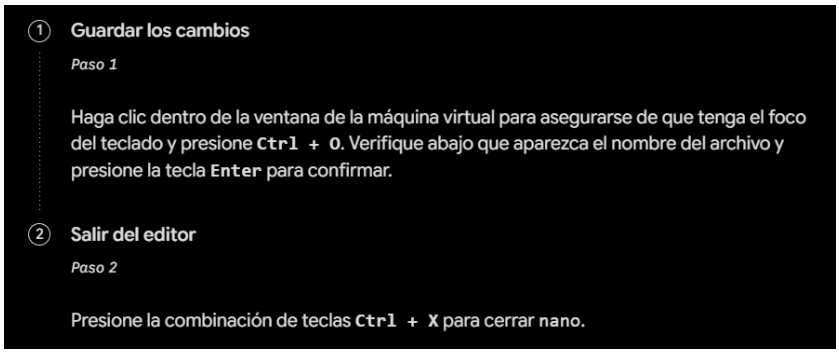
**¿Qué es un `Target` y qué significa `multi-user.target`?**
En Linux, el proceso de encendido no ocurre de golpe; se realiza por **etapas de preparación** llamadas `Targets` (Metas/Objetivos).
Piense en la apertura diaria de un restaurante:

1. **Etapa 1 (`sysinit.target`)**: Encender las luces y abrir la puerta principal del local.
2. **Etapa 2 (`network.target`)**: Conectar la caja registradora a Internet y encender las cocinas.
3. **Etapa 3 (`multi-user.target`)**: Abrir las puertas a los clientes. El restaurante está funcionando al 100% y listo para recibir múltiples personas a la vez.

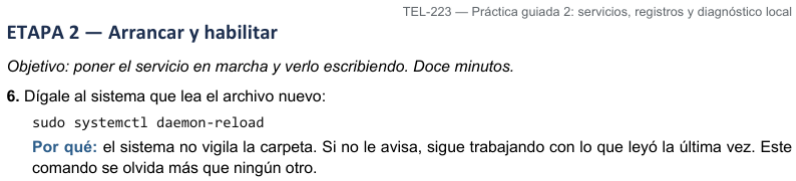
- `multi-user.target`: Es la "Etapa 3" en los servidores Linux. Representa el estado de trabajo normal donde la máquina ya cargó el disco, la red y la memoria, quedando lista para recibir usuarios y ejecutar servicios.

**¿Qué hace `WantedBy=multi-user.target`?**
La directiva `WantedBy` es la lista de deseos de una etapa.
Al escribir `WantedBy=multi-user.target` dentro de su archivo `.service`, le está diciendo al sistema: *"Guarda un acceso directo de este servicio en la carpeta de la Etapa 3"*. De este modo, cuando el servidor encienda y llegue al estado normal de trabajo (`multi-user.target`), revisará esa lista de deseos y arrancará su servicio `tel223-latido` de forma automática.
**En resumen:**

> • `Type=simple`: Arranca el programa directamente sin esperar confirmaciones.
> • `WantedBy=multi-user.target`: Engancha el programa al arranque automático cuando el equipo llega a su modo de trabajo normal.




# Etapa 2






Este comando obliga al sistema operativo a volver a leer los archivos de configuración de servicios para que reconozca cualquier archivo nuevo o cambio recien guardado.

- `systemctl` **(*SYSTEM ConTroL* / Controlador del Sistema):** Viene de la unión de `system` (sistema) y `ctl` (abreviatura de *control*). Es la herramienta de terminal que funciona como un "control remoto" para enviarle órdenes directas a `systemd` (el Administrador Maestro).
- `daemon` **(Demonio / Proceso en segundo plano):** En informática, un *daemon* o demonio es un programa que trabaja en silencio en segundo plano sin necesidad de que un usuario lo esté mirando. En este comando, la palabra se refiere al proceso maestro de `systemd` que administra la máquina.
- `reload` **(Recargar / Actualizar memoria):** Es la acción concreta de *"recargar"*. Le exige a `systemd` que vuelva a escanear las carpetas del disco duro (como `/etc/systemd/system/`) y memorice los cambios o nuevos archivos `.service` que haya creado.

## ¿Por qué es obligatorio ejecutarlo?

Linux no pierde tiempo ni recursos vigilando la carpeta de servicios a cada segundo. Cuando usted crea el archivo `tel223-latido.service`, el archivo ya existe en el disco duro, pero `systemd` **todavía no sabe que está ahí** porque solo consulta su memoria interna.

**La analogía de la secretaria:**

Imagine que le deja una carta sobre el escritorio a una secretaria mientras ella trabaja mirando su pantalla. La carta está allí, pero ella no la atenderá hasta que usted le avise *"revisa tu escritorio"*. El comando `daemon-reload` es ese aviso explícito para que el sistema reconozca la nueva instrucción.

## Cómo explicarlo en sus propias palabras:

> "El comando `systemctl` (control del sistema) le exige al proceso maestro (`daemon`) que recargue (`reload`) su memoria para que se entere de que creamos un nuevo archivo de servicio en el disco."



`systemctl` es la herramienta de terminal que sirve como panel de control para comunicarse con `systemd` (el supervisor que trabaja en segundo plano).

**La analogía de la Televisión Inteligente**

- `systemd` **es el sistema operativo interno del televisor:** Es el cerebro electrónico que mantiene la pantalla encendida, procesa el video y conecta el sonido. Trabaja solo y no tiene botones físicos en la superficie.
- `systemctl` **es el control remoto que tienes en la mano:** Cada vez que presionas un botón del control remoto (como subir volumen o cambiar de canal), el control le envía una señal infrarroja al televisor.

Tú no puedes tocar el cerebro del televisor directamente con las manos; necesitas el control remoto (`systemctl`) para enviarle órdenes desde la comodidad de tu asiento (la terminal).

**¿Por qué existen por separado?**

En Linux, los programas que administran el sistema corren de forma "invisible" en segundo plano (se les llama *daemons* o demonios). Como

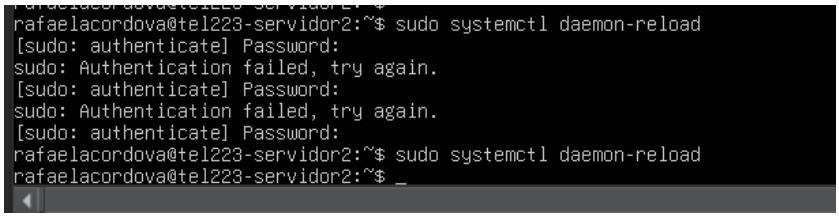
no tienen una ventana gráfica donde hacer clic, `systemctl` es la herramienta puente que traduce el texto simple que tecleas a una orden técnica que el supervisor `systemd` entiende.

**Los "botones" más importantes de este control remoto**

- `systemctl start nombre-servicio` **(Botón Play):** Le ordena a `systemd` que ponga en marcha el servicio en ese preciso instante.
- `systemctl status nombre-servicio` **(Botón de Información):** Le pide a `systemd` que te muestre en pantalla un informe detallado sobre si el servicio está corriendo o si se rompió.
- `systemctl enable nombre-servicio` **(Botón de Programación):** Le ordena a `systemd` que guarde ese servicio en la memoria de encendido para que arranque solo la próxima vez que enciendas la computadora.
- `systemctl daemon-reload` **(Botón de Recargar/Refrescar):** Le avisa a `systemd` que leíste o creaste un archivo nuevo en el disco y que debe actualizar su lista de instrucciones.

**En resumen para explicar a otra persona:**

> " No podemos hablarle a `systemd` directamente porque es un proceso fantasma en segundo plano. Usamos `systemctl` como si fuera un control remoto con botones específicos (`start`, `status`, `enable`, `daemon-reload`) para ordenarle al sistema qué hacer con nuestros servicios."




**7.** Arranque el servicio y consulte su estado:

```
sudo systemctl start tel223-latido
systemctl status tel223-latido
```

**Qué debe ver:** una línea que dice active (running), la fecha desde la que está activo y, abajo, las últimas líneas que escribió. Si la salida llena la pantalla, salga con la tecla q.

```
rafaelacordova@tel223-servidor2:~$ sudo systemctl start tel223-latido
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-latido
● tel223-latido.service - Servicio de latido TEL-223
     Loaded: loaded (/etc/systemd/system/tel223-latido.service; disabled; preset: enabled)
     Active: active (running) since Mon 2026-09-07 01:34:22 UTC; 16s ago
 Invocation: fa4a58d315ed49bfa7600ac2d4d48e9f
   Main PID: 2060 (latido.sh)
      Tasks: 2 (limit: 1882)
     Memory: 1.9M (peak: 2.6M)
        CPU: 163ms
     CGroup: /system.slice/tel223-latido.service
             ├─2060 /bin/bash /usr/local/bin/latido.sh
             └─2072 sleep 5

sep 07 01:34:22 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:34:22 tel223-servidor2 latido.sh[2060]: latido 01:34:22
sep 07 01:34:27 tel223-servidor2 latido.sh[2060]: latido 01:34:27
sep 07 01:34:32 tel223-servidor2 latido.sh[2060]: latido 01:34:32
sep 07 01:34:37 tel223-servidor2 latido.sh[2060]: latido 01:34:37
rafaelacordova@tel223-servidor2:~$ _
```

[con respecto a las últimas líneas que escribió se refiere a los últimos 3 registros de la pantalla]

**8.** Haga que arranque solo al encender el servidor:

```
sudo systemctl enable tel223-latido
```

**Qué acaba de hacer:** hasta ahora el servicio corría, pero desaparecía en el próximo reinicio. Con este comando queda enabled: volverá solo.

```
rafaelacordova@tel223-servidor2:~$ sudo systemctl enable tel223-latido
Created symlink '/etc/systemd/system/multi-user.target.wants/tel223-latido.service' → '/etc/systemd/system/tel223-latido.service'.
rafaelacordova@tel223-servidor2:~$
```

**9.** Vea su servicio escribiendo en vivo:

```
journalctl -u tel223-latido -f
```

**Qué debe ver:** una línea nueva cada cinco segundos. Ese es su servicio, funcionando.

**Para salir:** presione Control C. La terminal vuelve a responder.



Este comando le permite inspeccionar y monitorear en tiempo real la bitácora de mensajes (logs) que va generando exclusivamente su servicio.

`journalctl` **(*JOURNAL ConTroL* / Controlador del Diario de Registro)**

- **¿Qué hace?** Es el visor oficial de eventos y registros del sistema en Linux.
- **Explicación sencilla:** `systemd` tiene un "diario de campo" gigante donde anota todo lo que pasa en la computadora. `journalctl` es la herramienta que abre ese diario para que usted pueda leerlo.
- `u` **(*Unit* / Unidad de servicio)**
- **¿Qué hace?** Aplica un filtro para mostrar únicamente la información del servicio indicado.

- **Explicación sencilla:** El diario del sistema tiene millones de líneas de todos los programas. El parámetro `u` le ordena al visor: *"Ignora el resto del sistema y enséñame solo las anotaciones de esta unidad específica"*.

`tel223-latido` **(El nombre de la unidad)**

- **¿Qué hace?** Especifica el servicio exacto al cual queremos hacerle seguimiento.
- **Explicación sencilla:** Es el nombre del archivo de servicio que creamos previamente (`tel223-latido.service`). En Linux se puede escribir omitiendo la extensión `.service` al consultar registros.
- `f` **(*Follow* / Seguir en vivo)**
- **¿Qué hace?** Mantiene la pantalla "enganchada" actualizándose automáticamente cada vez que se genera un nuevo evento.
- **Explicación sencilla:** Sin `f`, el comando mostraría el historial pasado y se cerraría de inmediato. Con `f`, la terminal se queda abierta en modo "transmisión en vivo": cada vez que su script `latido.sh` imprima la palabra *"latido"*, la orden la mostrará en pantalla al instante.

**La analogía del cuaderno de bitácora**
Imagine la bitácora de un guardia de seguridad nocturno:

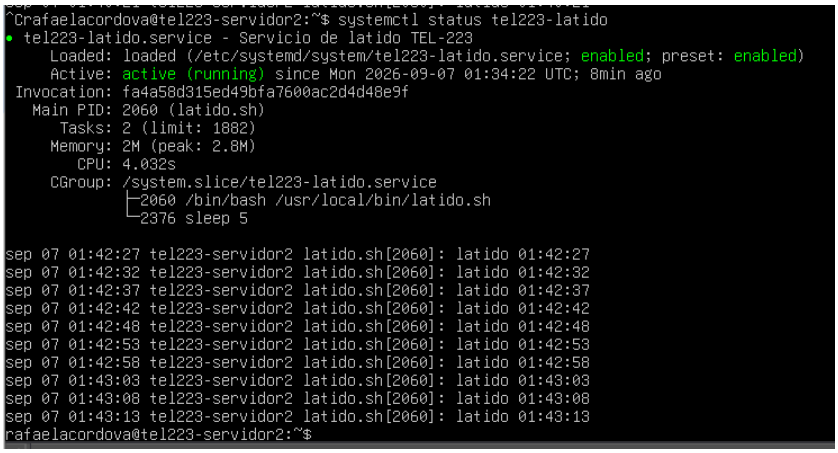
- `journalctl`: Es abrir el cuaderno del guardia.
- `u tel223-latido`: Es pedirle al guardia que busque únicamente las anotaciones referidas al empleado "tel223-latido".
- `f`: Es sentarse al lado del guardia a ver cómo escribe en vivo cada nueva novedad que ocurre minuto a minuto.

**Detalle práctico fundamental**
Al usar la opción `-f`, el comando **nunca se cerrará por sí solo** porque se queda esperando nuevos datos indefinidamente. Para detener el monitoreo y recuperar el control de su terminal, presione la combinación de teclas `Ctrl + C`.
**Cómo explicarlo en sus propias palabras:**

> *"El comando abre el visor de registros del sistema (`journalctl`), filtra únicamente las entradas pertenecientes a nuestra unidad (`-u tel223-latido`) y se queda siguiendo en vivo (`-f`) cada nuevo registro emitido por el script."*




| | | | |
|---|---|---|---|
| **El estado de mi servicio dice** | active | Está enabled | ☒ Sí ☐ No |
| **PUNTO DE CONTROL 1** — su servicio está en active (running) y escribe en el registro. Espere la verificación del docente. | | | |




# ETAPA 3

**ETAPA 3 — Falla A: la ruta equivocada**

*Objetivo: romper el servicio a propósito y descubrir por qué falló. Quince minutos.*

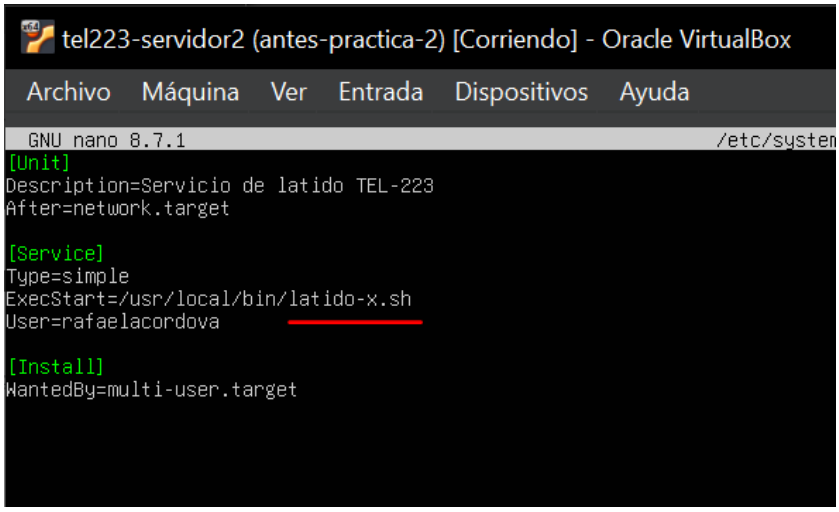
**Regla del diagnóstico:** antes de cada comando escriba en su bitácora qué espera ver. Diagnosticar no es probar comandos hasta que algo funcione: es formular una hipótesis y comprobarla.

**10.** Edite el archivo de unidad y cambie la ruta de ExecStart por una que no existe:

```
sudo nano /etc/systemd/system/tel223-latido.service
```

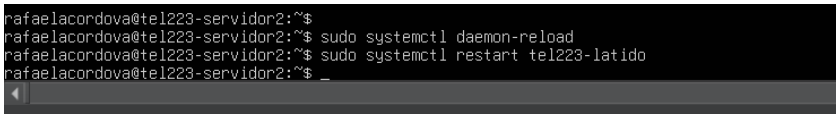
Reemplace esa línea por:

```
ExecStart=/usr/local/bin/latido-x.sh
```




**11.** Recargue y reinicie. Antes de mirar el resultado, escriba su hipótesis en la bitácora.

```
sudo systemctl daemon-reload
sudo systemctl restart tel223-latido
```






Para cambiar un servicio en ejecución se requiere un flujo de dos pasos: primero actualizar la memoria del Administrador Maestro (`daemon-reload`) y luego aplicar la orden al programa en segundo plano (`restart`).

**Por qué son necesarios dos comandos separados**

`systemd` mantiene la configuración de todos los servicios cargada en la memoria RAM para responder con rapidez. Por esa razón, modificar el archivo en el disco no afecta directamente lo que está corriendo:

- `sudo systemctl daemon-reload` **(Actualiza la memoria del sistema):** Le ordena a `systemd` releer el archivo `/etc/systemd/system/tel223-latido.service` desde el disco duro. Sin este comando, `systemd` sigue

usando la versión previa guardada en su memoria RAM y no se entera del cambio de ruta.

- `sudo systemctl restart tel223-latido` **(Aplica el cambio al programa):** Detiene el proceso actual en ejecución y arranca una instancia nueva aplicando la instrucción modificada. El comando `daemon-reload` por sí solo no apaga ni reinicia programas, únicamente refresca la lista de reglas.

**Hipótesis para tu bitácora (Etapa 3 - Falla A)**
En la sección de la bitácora de tu guía, en la pregunta *«¿Qué esperaba que dijera el estado antes de mirarlo?»*, debes escribir una predicción técnica como esta:

> "Espero que el estado cambie de `active` a `failed`. Como la ruta `/usr/local/bin/latido-x.sh` no existe en el disco duro, `systemd` intentará invocar el programa, no encontrará el archivo ejecutable y el servicio colapsará al intentar iniciar, arrojando un código de error de ejecución".

**El motivo de provocar fallas a propósito en la guía**
La guía indica explícitamente que el objetivo de la sesión no es lograr que el script funcione, sino aprender a diagnosticar por qué dejó de funcionar.

- **Diferencia entre QUÉ pasó y POR QUÉ pasó:** El comando `systemctl status` solo muestra el estado superficial (**QUÉ** ocurrió: el servicio dice `failed`). El visor de registros `journalctl` muestra el detalle interno (**POR QUÉ** ocurrió: *"No tal fichero o directorio"*).
- **El peligro de los códigos engañosos:** En la práctica comprobarás que dos problemas totalmente distintos (una ruta que no existe versus un archivo sin permisos de ejecución) producen el mismo código de error en `status`. Romper el servicio a propósito te enseña a no adivinar soluciones a ciegas, sino a confirmar la causa leyendo la bitácora de registros.
- **Preparación para el examen:** En el examen parcial no se evalúa si el servicio está activo, sino tu capacidad de explicar la causa exacta de

la falla cuando el docente altere la configuración de tu servidor.

¿Por qué ese orden?

💡 Si ejecutara los comandos al revés (`restart` y luego `daemon-reload`), **el cambio no surtiría efecto** al momento de reiniciar el servicio.

## Qué ocurre paso a paso si invierte el orden

- **1. Si ejecuta `restart` primero:** `systemd` apaga el servicio y lo vuelve a encender utilizando **la configuración antigua** que aún conserva en su memoria RAM. Como el sistema no consulta el disco duro durante un reinicio común, ignora los cambios que acaba de editar en el archivo `.service`.
- **2. Si ejecuta `daemon-reload` después:** `systemd` finalmente lee el archivo modificado del disco y actualiza su memoria RAM con la nueva regla. Sin embargo, **el programa ya fue arrancado en el paso anterior con las instrucciones viejas**, por lo que tendría que ejecutar un segundo `restart` para que recién aplique el cambio.

## La analogía de la receta de cocina

Imagine a un cocinero que memorizó una receta de sopa en su cabeza (`systemd` guardando la configuración en memoria RAM):

1. Usted borra "sal" y escribe "pimienta" en la hoja de papel sobre la mesa (modifica el archivo `.service` en el disco).
2. **Si le grita "¡Haz la sopa de nuevo!" (`restart`) antes de avisarle:** El cocinero preparará el plato usando su memoria vieja (con sal).
3. **Si después le dice "¡Lee la nueva receta sobre la mesa!" (`daemon-reload`):** El cocinero memoriza la receta con pimienta para el futuro, pero el plato que acaba de servir en la mesa ya fue cocinado con sal.

Para que la sopa salga con pimienta en ese instante, primero debe obligarlo a leer la hoja (`daemon-reload`) y luego pedirle que cocine (`restart`).

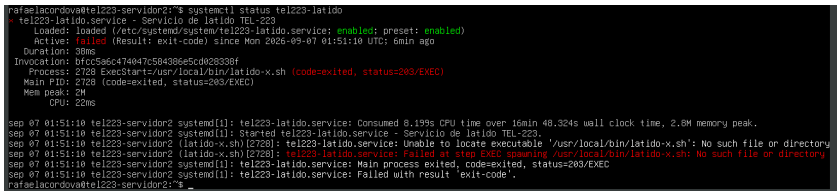
## Cómo verificar si cometió este error

Si olvida hacer el `daemon-reload` e intenta reiniciar o consultar el estado, la terminal mostrará un mensaje de advertencia como este:
`Warning: The unit file, source configuration file or drop-ins of tel223-latido.service changed on disk. Run 'systemctl daemon-reload' to reload units.`
Esa alerta confirma que el servidor detectó que el archivo en el disco cambió, pero que el servicio activo sigue corriendo con la versión vieja porque no se recargó la memoria.

**12.** Ahora sí, consulte el estado:

```
systemctl status tel223-latido
```

**Qué debe ver:** una línea que dice failed y un código de error. Anote el código exacto en su bitácora.




💡 La captura muestra el colapso controlado del servicio tras haber modificado la ruta del script en el archivo de unidad, ofreciendo los datos exactos necesarios para responder a la bitácora.

**Análisis de la Ficha Técnica (El "QUÉ pasó")**

| Línea en Pantalla | Valor Exacto | Significado Técnico |
|---|---|---|
| `Loaded:` | `enabled` | El archivo `.service` sigue estando configurado para arrancar automáticamente al encender el equipo. |
| `Active:` | `failed (Result: exit-code)` | El servicio intentó iniciar, pero colapsó inmediatamente y se detuvo. |
| `Process:` | `status=203/EXEC` | **Código numérico de error:** Significa que `systemd` no pudo ejecutar la orden configurada en la línea `ExecStart`. |

**Análisis del Registro Integrado (El "POR QUÉ pasó")**

En las últimas líneas rojas de la pantalla, `systemd` incluye el diagnóstico extraído del diario de eventos:

- `Unable to locate executable '/usr/local/bin/latido-x.sh'`: El sistema informa que buscó el programa en el disco, pero la ruta no existe.
- `No such file or directory`: Es la causa raíz devuelta por Linux al no hallar el archivo `latido-x.sh`.

**Cómo llenar la Bitácora de la Falla A**
Con esta pantalla en mano, puede completar las casillas de la bitácora con los datos exactos:

- **¿Qué dijo el estado? (código exacto):** `failed (Result: exit-code)` o `status=203/EXEC`.
- **¿Qué dijo el registro? (frase textual):** `Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory`.
- **¿Cuál era la causa?:** Se configuró una ruta inexistente (`/usr/local/bin/latido-x.sh`) en la directiva `ExecStart` del archivo `.service`.
- **¿Cómo la corregí?:** Editando el archivo de unidad con `nano` para restaurar la ruta real (`/usr/local/bin/latido.sh`), ejecutando `sudo systemctl daemon-reload` y luego `sudo systemctl restart tel223-latido`.
- **¿Cómo verifiqué que quedó corregida?:** Al ejecutar `systemctl status tel223-latido`, la línea `Active:` volvió a mostrar `active (running)`.



El código **203** (`status=203/EXEC`) es un número de error interno de `systemd` llamado `EXIT_EXEC`. Significa que el sistema preparó el entorno para el servicio, pero la orden enviada a la terminal no se pudo ejecutar en el disco duro.
**1.** `(Result: exit-code)`
Indica la **forma** en que finalizó el servicio:

- `exit-code`**:** Significa que el intento de ejecución terminó entregando un código numérico de error.

- **Para qué sirve:** Permite distinguir si un servicio colapsó por un error interno (`exit-code`), si un administrador lo mató a la fuerza (`Result: signal`), o si el programa no respondió a tiempo (`Result: timeout`).

**2. `code=exited` en el `Main PID`**

- `Main PID: 2728`**:** Es el Identificador de Proceso (*Process ID*) que Linux le asignó en la memoria RAM a esa orden en particular.
- `code=exited`**:** Indica la forma en que cerró el proceso. Significa que el subproceso finalizó llamando al cierre estándar del sistema (`exit`), a diferencia de `code=killed`, que aparece cuando un proceso es destruido bruscamente desde fuera.

**3. Desglose de `Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory`**

Esta línea es el mensaje completo de diagnóstico del núcleo de Linux:

- `Failed at step EXEC`**:** *"Falló en la etapa de ejecución"*. Confirma que el sistema creó el subproceso correctamente, pero colapsó al invocar la orden asignada en `ExecStart`.
- `spawning`**:** *"Lanzando / Engendrando"*. Es el término técnico que usa Linux cuando `systemd` intenta invocar y poner en marcha un nuevo programa en la memoria.
- `/usr/local/bin/latido-x.sh`**:** Muestra la ruta física exacta a la que `systemd` intentó acceder.
- `: No such file or directory`**:** La causa de fondo devuelta por el sistema operativo. Explica que el archivo buscado no existe en esa carpeta del disco.

**En resumen para la bitácora:**

El estado `failed` te dice que la fase de ejecución falló (`step EXEC`) devolviendo el código 203; el texto `: No such file or directory` te confirma que la causa fue escribir una ruta de archivo inexistente.

FLUJO DE TRABAJO DEL ERROR 203:

`systemd` separa el proceso en dos etapas independientes: **Análisis de Sintaxis** (lectura del texto) y **Ejecución Real** (acceso al disco duro).

**Fase 1: Lectura e Interpretación (Sintaxis)**

Cuando ejecutas `systemctl daemon-reload`, `systemd` lee el archivo de texto `.service` y valida únicamente la estructura sintáctica:

- **Valida las secciones y claves:** Confirma que entiende las etiquetas entre corchetes (`[Unit]`, `[Service]`, `[Install]`) y los nombres de las directivas (`ExecStart`, `User`, `Description`).
- **Valida el formato:** Comprueba que cada asignación tenga el formato correcto `Propiedad=Valor`.
- **Almacena la instrucción:** Guarda la cadena de texto `ExecStart=/usr/local/bin/latido-x.sh` en la memoria RAM como una simple orden pendiente de ejecución.

En esta fase, `systemd` **no verifica la existencia del archivo en el disco**. No realiza consultas al sistema de archivos para comprobar si la ruta es real ni si existen permisos de ejecución.

**Fase 2: El paso a la Etapa `EXEC` (Ejecución)**

El acceso al disco duro y la comprobación del ejecutable ocurren únicamente cuando se envía la orden de arranque (`systemctl start` o `restart`):

1. **Creación del entorno:** `systemd` solicita al núcleo de Linux (*kernel*) la creación de un nuevo proceso en la memoria RAM (asignando un PID).
2. **Llamada a la ejecución (`EXEC`):** Dentro de ese nuevo proceso, `systemd` invoca la función del sistema operativo encargada de cargar programas (`execve`) pasándole la ruta `/usr/local/bin/latido-x.sh`.
3. **Respuesta del Kernel:** El núcleo de Linux busca la ruta en la estructura física del disco duro. Al no encontrar la entrada en el directorio, el núcleo aborta la operación y devuelve la llamada de error estándar del sistema `ENOENT` (*No such file or directory*).
4. **Captura del error:** `systemd` captura el fallo en esa etapa exacta, mapea la respuesta del núcleo al código interno `203/EXEC`, detiene el

proceso y marca la unidad en estado `failed`.

**¿Por qué está diseñado así?**
En servidores Linux, un servicio puede depender de carpetas compartidas en red o discos secundarios que tardan unos segundos en montarse durante el arranque del equipo. Si `systemd` exigiera que el archivo ejecutable estuviera presente en el disco desde el momento en que lee la configuración, el sistema operativo generaría errores falsos durante el inicio.

**13.** El estado le dice QUE falló. Ahora averigüe POR QUÉ:

```
journalctl -u tel223-latido -n 20
```

**Qué debe buscar:** un mensaje que explique qué pasó al intentar ejecutar el programa. Anótelo textualmente en la bitácora: esa frase es la causa.



```
rafaelacordova@tel223-servidor2:~$ journalctl -u tel223-latido -n 20
sep 07 01:50:10 tel223-servidor2 latido.sh[2060]: latido 01:50:10
sep 07 01:50:19 tel223-servidor2 latido.sh[2060]: latido 01:50:19
sep 07 01:50:24 tel223-servidor2 latido.sh[2060]: latido 01:50:24
sep 07 01:50:29 tel223-servidor2 latido.sh[2060]: latido 01:50:29
sep 07 01:50:34 tel223-servidor2 latido.sh[2060]: latido 01:50:34
sep 07 01:50:39 tel223-servidor2 latido.sh[2060]: latido 01:50:39
sep 07 01:50:45 tel223-servidor2 latido.sh[2060]: latido 01:50:45
sep 07 01:50:50 tel223-servidor2 latido.sh[2060]: latido 01:50:50
sep 07 01:50:55 tel223-servidor2 latido.sh[2060]: latido 01:50:55
sep 07 01:51:00 tel223-servidor2 latido.sh[2060]: latido 01:51:00
sep 07 01:51:07 tel223-servidor2 latido.sh[2060]: latido 01:51:07
sep 07 01:51:10 tel223-servidor2 systemd[1]: Stopping tel223-latido.service - Servicio de latido TEL-223...
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Deactivated successfully.
sep 07 01:51:10 tel223-servidor2 systemd[1]: Stopped tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Consumed 8.199s CPU time over 16min 48.324s wall clock time, 2.8M memory peak.
sep 07 01:51:10 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:51:10 tel223-servidor2 (latido-x.sh)[2728]: tel223-latido.service: Unable to locate executable '/usr/local/bin/latido-x.sh': No such file or directory
sep 07 01:51:10 tel223-servidor2 (latido-x.sh)[2728]: tel223-latido.service: Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Main process exited, code=exited, status=203/EXEC
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Failed with result 'exit-code'.
rafaelacordova@tel223-servidor2:~$
```

**BITÁCORA DE DIAGNÓSTICO**

*Esta bitácora es la evidencia de la sesión. Complétela durante la práctica, no al final.*

**Falla A — la ruta equivocada**

| Pregunta | Mi respuesta |
|---|---|
| ¿Qué esperaba que dijera el estado antes de mirarlo? | Espero que pare de decir running y cambie a failed, porque systemd ya no encuentra la ubicación real del archivo en el disco |
| ¿Qué dijo el estado? (código exacto) | failed (Result: exit-code) status=203/EXEC -> Result indica que fue un error causado dentro del servidor |
| ¿Qué dijo el registro? (frase textual) | tel223-latifo.service: Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory |
| ¿Cuál era la causa? | El llenado en el campo ExecStart de un archivo o ruta de archivo inexistente |
| ¿Cómo la corregí? | |
| ¿Cómo verifiqué que quedó corregida? | |



**14.** Corrija la ruta, recargue y verifique que volvió a funcionar:

```
sudo nano /etc/systemd/system/tel223-latido.service
sudo systemctl daemon-reload
sudo systemctl restart tel223-latido
systemctl status tel223-latido
```

**PUNTO DE CONTROL 2 —** anotó el código de error, el mensaje del registro y la causa de la falla A.

```
  GNU nano 8.7.1
[Unit]
Description=Servicio de latido TEL-223
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/latido.sh
User=rafaelacordova

[Install]
WantedBy=multi-user.target
```

```
rafaelacordova@tel223-servidor2:~$
rafaelacordova@tel223-servidor2:~$ sudo systemctl daemon-reload
rafaelacordova@tel223-servidor2:~$ sudo systemctl restart tel223-latido
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-latido
● tel223-latido.service - Servicio de latido TEL-223
     Loaded: loaded (/etc/systemd/system/tel223-latido.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-09-07 02:31:18 UTC; 11s ago
 Invocation: a4352336c3a0470e81bfba8c943323cc
   Main PID: 2899 (latido.sh)
      Tasks: 2 (limit: 1882)
     Memory: 1.8M (peak: 2.5M)
        CPU: 191ms
     CGroup: /system.slice/tel223-latido.service
             ├─2899 /bin/bash /usr/local/bin/latido.sh
             └─2909 sleep 5

sep 07 02:31:18 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:31:18 tel223-servidor2 latido.sh[2899]: latido 02:31:18
sep 07 02:31:23 tel223-servidor2 latido.sh[2899]: latido 02:31:23
sep 07 02:31:29 tel223-servidor2 latido.sh[2899]: latido 02:31:29
rafaelacordova@tel223-servidor2:~$ _
```

# Etapa 4

### ETAPA 4 — Falla B: el permiso que falta

*Objetivo: provocar una segunda falla y compararla con la primera. Quince minutos. Es la parte más importante del día.*

**15.** Con el servicio ya funcionando, quítele el permiso de ejecución al script:

```
sudo chmod -x /usr/local/bin/latido.sh
sudo systemctl restart tel223-latido
```

```
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-latido
× tel223-latido.service - Servicio de latido TEL-223
     Loaded: loaded (/etc/systemd/system/tel223-latido.service; enabled; preset: enabled)
     Active: failed (Result: exit-code) since Mon 2026-09-07 02:40:03 UTC; 2min 24s ago
   Duration: 25ms
 Invocation: b6e3ec125a4b44839f12962023167042
    Process: 3158 ExecStart=/usr/local/bin/latido.sh (code=exited, status=203/EXEC)
   Main PID: 3158 (code=exited, status=203/EXEC)
   Mem peak: 1.9M
        CPU: 16ms

sep 07 02:40:03 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Unable to locate executable '/usr/local/bin/latido.sh': Permission denied
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Failed at step EXEC spawning /usr/local/bin/latido.sh: Permission denied
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Main process exited, code=exited, status=203/EXEC
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Failed with result 'exit-code'.
rafaelacordova@tel223-servidor2:~$ _
```

**16.** Antes de mirar: escriba en la bitácora qué código de error espera encontrar.

**17.** Consulte el estado:

```
systemctl status tel223-latido
```

> **Deténgase aquí —** compare este código con el que anotó en la falla A. ¿Es el mismo? Anote la respuesta antes de seguir.

```
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-latido
× tel223-latido.service - Servicio de latido TEL-223
     Loaded: loaded (/etc/systemd/system/tel223-latido.service; enabled; preset: enabled)
     Active: failed (Result: exit-code) since Mon 2026-09-07 02:40:03 UTC; 2min 24s ago
   Duration: 25ms
 Invocation: b6e3ec125a4b44839f12962023167042
    Process: 3158 ExecStart=/usr/local/bin/latido.sh (code=exited, status=203/EXEC)
   Main PID: 3158 (code=exited, status=203/EXEC)
   Mem peak: 1.9M
        CPU: 16ms

sep 07 02:40:03 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Unable to locate executable '/usr/local/bin/latido.sh': Permission denied
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Failed at step EXEC spawning /usr/local/bin/latido.sh: Permission denied
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Main process exited, code=exited, status=203/EXEC
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Failed with result 'exit-code'.
rafaelacordova@tel223-servidor2:~$
```

**18.** Ahora lea el registro:

```
journalctl -u tel223-latido -n 20
```

**Qué debe ver:** un mensaje distinto al de la falla A, aunque el código de error sea el mismo. Anótelo textualmente. tel223-latido.service: Failed at step EXEC spawning /urs/local/bin/latido.sh: Permission denied

```
rafaelacordova@tel223-servidor2:~$ journalctl -u tel223-latido -n 20
sep 07 02:39:09 tel223-servidor2 latido.sh[2899]: latido 02:39:09
sep 07 02:39:15 tel223-servidor2 latido.sh[2899]: latido 02:39:15
sep 07 02:39:20 tel223-servidor2 latido.sh[2899]: latido 02:39:20
sep 07 02:39:25 tel223-servidor2 latido.sh[2899]: latido 02:39:25
sep 07 02:39:30 tel223-servidor2 latido.sh[2899]: latido 02:39:30
sep 07 02:39:35 tel223-servidor2 latido.sh[2899]: latido 02:39:35
sep 07 02:39:40 tel223-servidor2 latido.sh[2899]: latido 02:39:40
sep 07 02:39:45 tel223-servidor2 latido.sh[2899]: latido 02:39:45
sep 07 02:39:50 tel223-servidor2 latido.sh[2899]: latido 02:39:50
sep 07 02:39:55 tel223-servidor2 latido.sh[2899]: latido 02:39:55
sep 07 02:40:00 tel223-servidor2 latido.sh[2899]: latido 02:40:00
sep 07 02:40:03 tel223-servidor2 systemd[1]: Stopping tel223-latido.service - Servicio de latido TEL-223...
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Deactivated successfully.
sep 07 02:40:03 tel223-servidor2 systemd[1]: Stopped tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Consumed 3.856s CPU time over 8min 45.454s wall clock time, 2.8M memory peak.
sep 07 02:40:03 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Unable to locate executable '/usr/local/bin/latido.sh': Permission denied
sep 07 02:40:03 tel223-servidor2 (latido.sh)[3158]: tel223-latido.service: Failed at step EXEC spawning /usr/local/bin/latido.sh: Permission denied
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Main process exited, code=exited, status=203/EXEC
sep 07 02:40:03 tel223-servidor2 systemd[1]: tel223-latido.service: Failed with result 'exit-code'.
rafaelacordova@tel223-servidor2:~$
```

**19.** Compruebe los permisos del script:

```
ls -l /usr/local/bin/latido.sh
```

**Qué debe ver:** las x desaparecieron. Ahí está la causa, confirmada por dos vías distintas: el registro y los permisos.

```
rafaelacordova@tel223-servidor2:~$ ls -l /usr/local/bin/latido.sh
-rw-r--r-- 1 root root 76 Sep  7 00:13 /usr/local/bin/latido.sh
rafaelacordova@tel223-servidor2:~$ _
```

# Etapa 5

## ETAPA 5 — Corregir y verificar

*Objetivo: dejar el servicio funcionando y revisar la historia completa. Diez minutos.*

**20.** Restituya el permiso y reinicie:

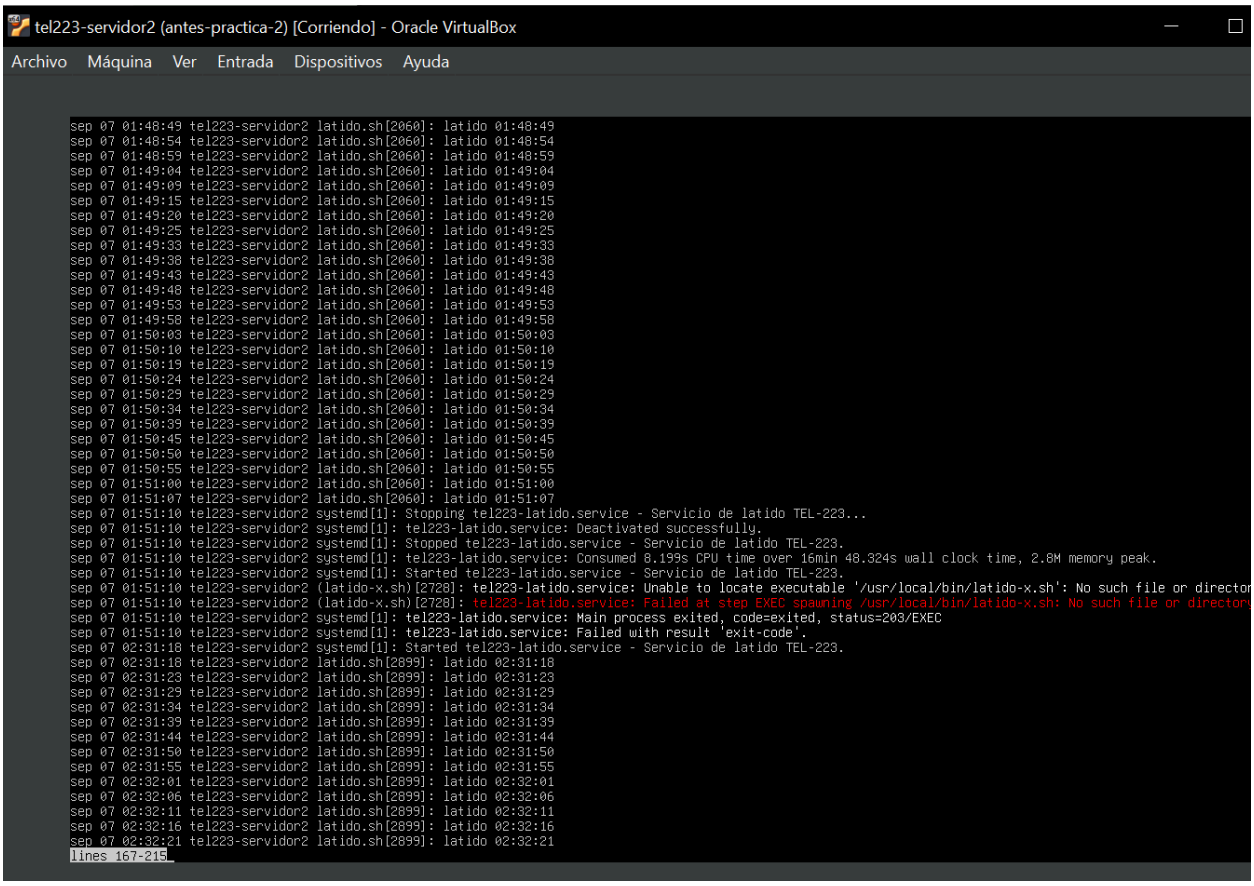
```
sudo chmod +x /usr/local/bin/latido.sh
sudo systemctl restart tel223-latido
systemctl status tel223-latido
```

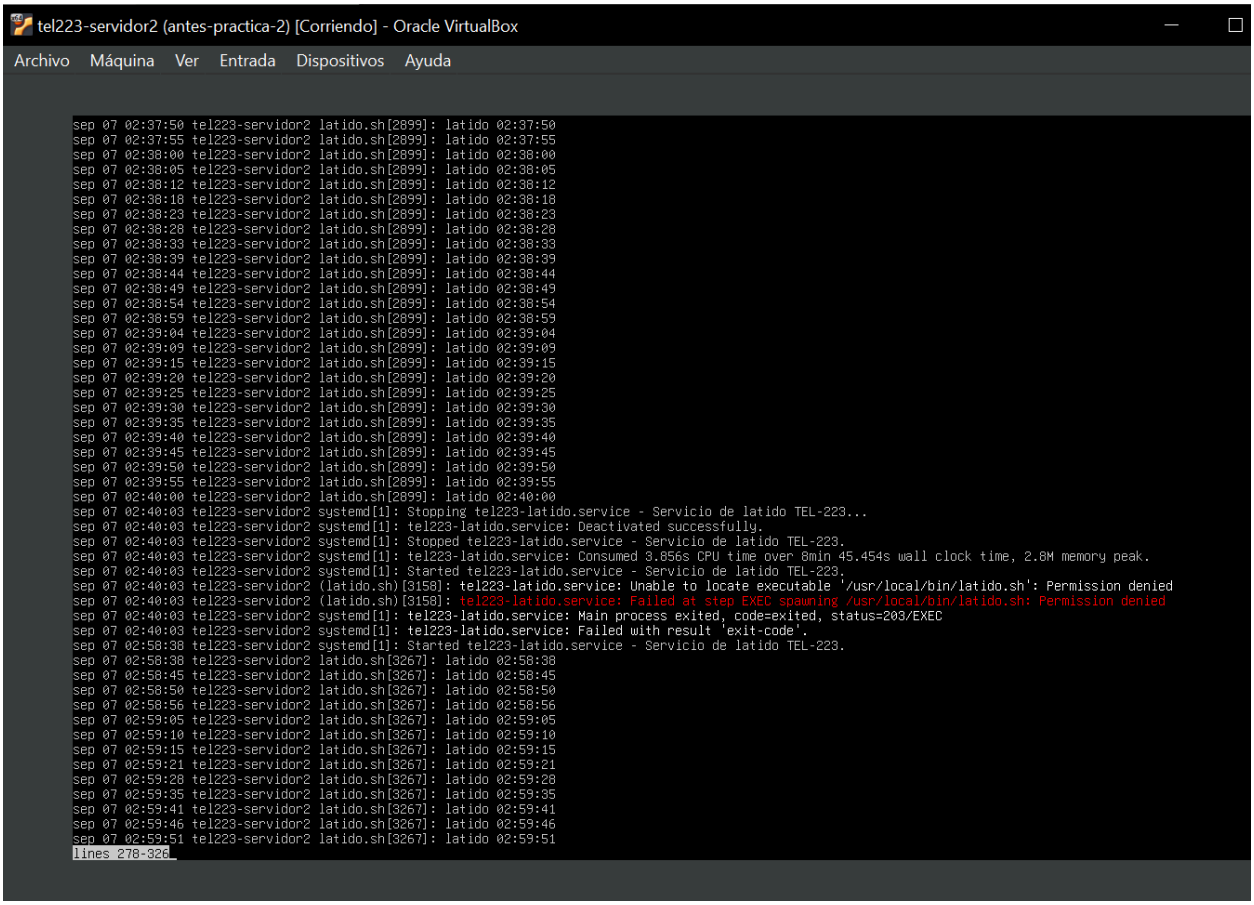
**Qué debe ver:** active (running) otra vez.

```
rafaelacordova@tel223-servidor2:~$ sudo chmod +x /usr/local/bin/latido.sh
[sudo: authenticate] Password:
rafaelacordova@tel223-servidor2:~$ sudo systemctl restart tel223-latido
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-latido
● tel223-latido.service - Servicio de latido TEL-223
     Loaded: loaded (/etc/systemd/system/tel223-latido.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-09-07 02:58:38 UTC; 18s ago
 Invocation: 9c2969c9dc5d49028b667ee41cc5dfa9
   Main PID: 3267 (latido.sh)
      Tasks: 2 (limit: 1882)
     Memory: 2M (peak: 2.5M)
        CPU: 131ms
     CGroup: /system.slice/tel223-latido.service
             ├─3267 /bin/bash /usr/local/bin/latido.sh
             └─3279 sleep 5

sep 07 02:58:38 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:58:38 tel223-servidor2 latido.sh[3267]: latido 02:58:38
sep 07 02:58:45 tel223-servidor2 latido.sh[3267]: latido 02:58:45
sep 07 02:58:50 tel223-servidor2 latido.sh[3267]: latido 02:58:50
sep 07 02:58:56 tel223-servidor2 latido.sh[3267]: latido 02:58:56
rafaelacordova@tel223-servidor2:~$
```

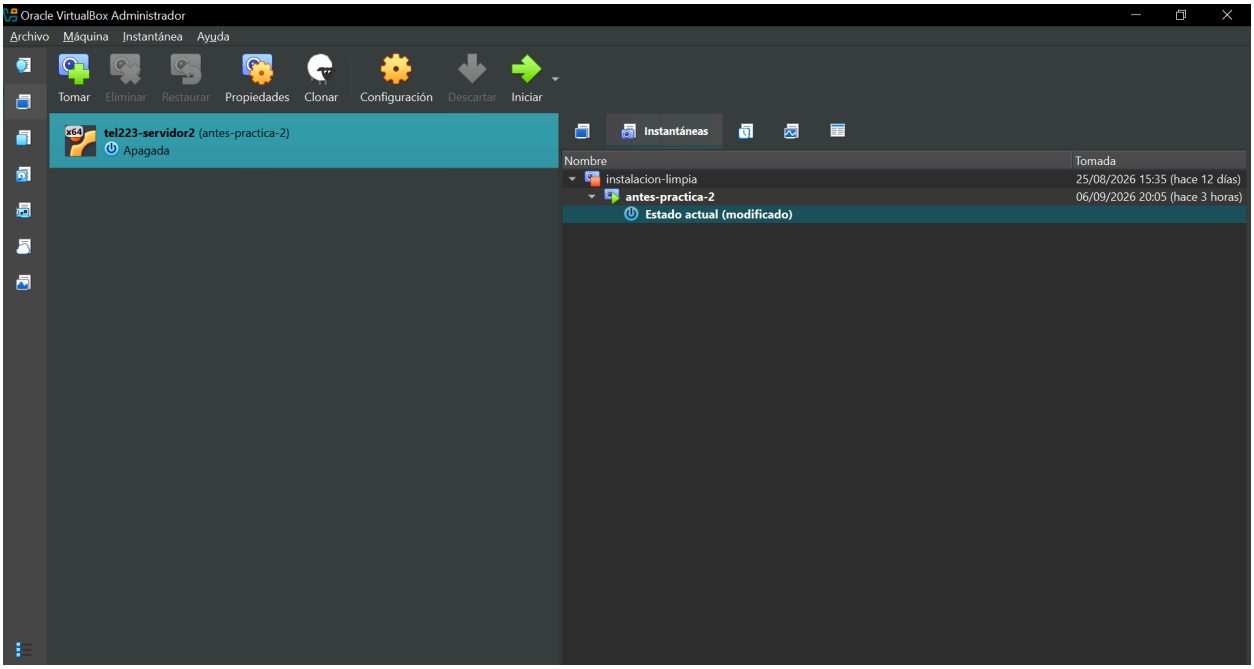
```
sep 07 01:34:22 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:34:22 tel223-servidor2 latido.sh[2060]: latido 01:34:22
sep 07 01:34:27 tel223-servidor2 latido.sh[2060]: latido 01:34:27
sep 07 01:34:32 tel223-servidor2 latido.sh[2060]: latido 01:34:32
sep 07 01:34:37 tel223-servidor2 latido.sh[2060]: latido 01:34:37
sep 07 01:34:42 tel223-servidor2 latido.sh[2060]: latido 01:34:42
sep 07 01:34:47 tel223-servidor2 latido.sh[2060]: latido 01:34:47
sep 07 01:34:52 tel223-servidor2 latido.sh[2060]: latido 01:34:52
sep 07 01:34:57 tel223-servidor2 latido.sh[2060]: latido 01:34:57
sep 07 01:35:02 tel223-servidor2 latido.sh[2060]: latido 01:35:02
sep 07 01:35:07 tel223-servidor2 latido.sh[2060]: latido 01:35:07
sep 07 01:35:13 tel223-servidor2 latido.sh[2060]: latido 01:35:13
sep 07 01:35:18 tel223-servidor2 latido.sh[2060]: latido 01:35:18
sep 07 01:35:23 tel223-servidor2 latido.sh[2060]: latido 01:35:23
sep 07 01:35:28 tel223-servidor2 latido.sh[2060]: latido 01:35:28
sep 07 01:35:33 tel223-servidor2 latido.sh[2060]: latido 01:35:33
sep 07 01:35:38 tel223-servidor2 latido.sh[2060]: latido 01:35:38
sep 07 01:35:45 tel223-servidor2 latido.sh[2060]: latido 01:35:45
sep 07 01:35:54 tel223-servidor2 latido.sh[2060]: latido 01:35:54
sep 07 01:35:59 tel223-servidor2 latido.sh[2060]: latido 01:35:59
sep 07 01:36:04 tel223-servidor2 latido.sh[2060]: latido 01:36:04
sep 07 01:36:09 tel223-servidor2 latido.sh[2060]: latido 01:36:09
sep 07 01:36:18 tel223-servidor2 latido.sh[2060]: latido 01:36:18
sep 07 01:36:23 tel223-servidor2 latido.sh[2060]: latido 01:36:23
sep 07 01:36:28 tel223-servidor2 latido.sh[2060]: latido 01:36:28
sep 07 01:36:33 tel223-servidor2 latido.sh[2060]: latido 01:36:33
sep 07 01:36:38 tel223-servidor2 latido.sh[2060]: latido 01:36:38
sep 07 01:36:44 tel223-servidor2 latido.sh[2060]: latido 01:36:44
sep 07 01:36:49 tel223-servidor2 latido.sh[2060]: latido 01:36:49
sep 07 01:36:54 tel223-servidor2 latido.sh[2060]: latido 01:36:54
sep 07 01:36:59 tel223-servidor2 latido.sh[2060]: latido 01:36:59
sep 07 01:37:04 tel223-servidor2 latido.sh[2060]: latido 01:37:04
sep 07 01:37:09 tel223-servidor2 latido.sh[2060]: latido 01:37:09
sep 07 01:37:14 tel223-servidor2 latido.sh[2060]: latido 01:37:14
sep 07 01:37:19 tel223-servidor2 latido.sh[2060]: latido 01:37:19
sep 07 01:37:24 tel223-servidor2 latido.sh[2060]: latido 01:37:24
sep 07 01:37:30 tel223-servidor2 latido.sh[2060]: latido 01:37:30
sep 07 01:37:35 tel223-servidor2 latido.sh[2060]: latido 01:37:35
sep 07 01:37:42 tel223-servidor2 latido.sh[2060]: latido 01:37:42
sep 07 01:37:47 tel223-servidor2 latido.sh[2060]: latido 01:37:47
sep 07 01:37:52 tel223-servidor2 latido.sh[2060]: latido 01:37:52
sep 07 01:37:58 tel223-servidor2 latido.sh[2060]: latido 01:37:58
sep 07 01:38:03 tel223-servidor2 latido.sh[2060]: latido 01:38:03
sep 07 01:38:08 tel223-servidor2 latido.sh[2060]: latido 01:38:08
sep 07 01:38:14 tel223-servidor2 latido.sh[2060]: latido 01:38:14
sep 07 01:38:19 tel223-servidor2 latido.sh[2060]: latido 01:38:19
sep 07 01:38:24 tel223-servidor2 latido.sh[2060]: latido 01:38:24
sep 07 01:38:29 tel223-servidor2 latido.sh[2060]: latido 01:38:29
sep 07 01:38:34 tel223-servidor2 latido.sh[2060]: latido 01:38:34
lines 1-49
```


tel223-servidor2 (antes-practica-2) [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
sep 07 01:48:49 tel223-servidor2 latido.sh[2060]: latido 01:48:49
sep 07 01:48:54 tel223-servidor2 latido.sh[2060]: latido 01:48:54
sep 07 01:48:59 tel223-servidor2 latido.sh[2060]: latido 01:48:59
sep 07 01:49:04 tel223-servidor2 latido.sh[2060]: latido 01:49:04
sep 07 01:49:09 tel223-servidor2 latido.sh[2060]: latido 01:49:09
sep 07 01:49:15 tel223-servidor2 latido.sh[2060]: latido 01:49:15
sep 07 01:49:20 tel223-servidor2 latido.sh[2060]: latido 01:49:20
sep 07 01:49:25 tel223-servidor2 latido.sh[2060]: latido 01:49:25
sep 07 01:49:33 tel223-servidor2 latido.sh[2060]: latido 01:49:33
sep 07 01:49:38 tel223-servidor2 latido.sh[2060]: latido 01:49:38
sep 07 01:49:43 tel223-servidor2 latido.sh[2060]: latido 01:49:43
sep 07 01:49:48 tel223-servidor2 latido.sh[2060]: latido 01:49:48
sep 07 01:49:53 tel223-servidor2 latido.sh[2060]: latido 01:49:53
sep 07 01:49:58 tel223-servidor2 latido.sh[2060]: latido 01:49:58
sep 07 01:50:03 tel223-servidor2 latido.sh[2060]: latido 01:50:03
sep 07 01:50:10 tel223-servidor2 latido.sh[2060]: latido 01:50:10
sep 07 01:50:19 tel223-servidor2 latido.sh[2060]: latido 01:50:19
sep 07 01:50:24 tel223-servidor2 latido.sh[2060]: latido 01:50:24
sep 07 01:50:29 tel223-servidor2 latido.sh[2060]: latido 01:50:29
sep 07 01:50:34 tel223-servidor2 latido.sh[2060]: latido 01:50:34
sep 07 01:50:39 tel223-servidor2 latido.sh[2060]: latido 01:50:39
sep 07 01:50:45 tel223-servidor2 latido.sh[2060]: latido 01:50:45
sep 07 01:50:50 tel223-servidor2 latido.sh[2060]: latido 01:50:50
sep 07 01:50:55 tel223-servidor2 latido.sh[2060]: latido 01:50:55
sep 07 01:51:00 tel223-servidor2 latido.sh[2060]: latido 01:51:00
sep 07 01:51:07 tel223-servidor2 latido.sh[2060]: latido 01:51:07
sep 07 01:51:10 tel223-servidor2 systemd[1]: Stopping tel223-latido.service - Servicio de latido TEL-223...
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Deactivated successfully.
sep 07 01:51:10 tel223-servidor2 systemd[1]: Stopped tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Consumed 8.199s CPU time over 16min 48.324s wall clock time, 2.8M memory peak.
sep 07 01:51:10 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 01:51:10 tel223-servidor2 (latido-x.sh)[2728]: tel223-latido.service: Unable to locate executable '/usr/local/bin/latido-x.sh': No such file or director
sep 07 01:51:10 tel223-servidor2 (latido-x.sh)[2728]: tel223-latido.service: Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Main process exited, code=exited, status=203/EXEC
sep 07 01:51:10 tel223-servidor2 systemd[1]: tel223-latido.service: Failed with result 'exit-code'.
sep 07 02:31:18 tel223-servidor2 systemd[1]: Started tel223-latido.service - Servicio de latido TEL-223.
sep 07 02:31:18 tel223-servidor2 latido.sh[2899]: latido 02:31:18
sep 07 02:31:23 tel223-servidor2 latido.sh[2899]: latido 02:31:23
sep 07 02:31:29 tel223-servidor2 latido.sh[2899]: latido 02:31:29
sep 07 02:31:34 tel223-servidor2 latido.sh[2899]: latido 02:31:34
sep 07 02:31:39 tel223-servidor2 latido.sh[2899]: latido 02:31:39
sep 07 02:31:44 tel223-servidor2 latido.sh[2899]: latido 02:31:44
sep 07 02:31:50 tel223-servidor2 latido.sh[2899]: latido 02:31:50
sep 07 02:31:55 tel223-servidor2 latido.sh[2899]: latido 02:31:55
sep 07 02:32:01 tel223-servidor2 latido.sh[2899]: latido 02:32:01
sep 07 02:32:06 tel223-servidor2 latido.sh[2899]: latido 02:32:06
sep 07 02:32:11 tel223-servidor2 latido.sh[2899]: latido 02:32:11
sep 07 02:32:16 tel223-servidor2 latido.sh[2899]: latido 02:32:16
sep 07 02:32:21 tel223-servidor2 latido.sh[2899]: latido 02:32:21
lines 167-215




Ctrl + C para salir

Apagado de la máquina tras sudo poweroff:


Oracle VirtualBox Administrador
Archivo Máquina Instantánea Ayuda
Tomar Eliminar Restaurar Propiedades Clonar Configuración Descartar Iniciar
tel223-servidor2 (antes-practica-2)
Apagada
Instantáneas
Nombre
Tomada
instalacion-limpia
25/08/2026 15:35 (hace 12 días)
antes-practica-2
06/09/2026 20:05 (hace 3 horas)
Estado actual (modificado)

## BITÁCORA DE DIAGNÓSTICO

*Esta bitácora es la evidencia de la sesión. Complétela durante la práctica, no al final.*

### Falla A — la ruta equivocada

| Pregunta | Mi respuesta |
|---|---|
| ¿Qué esperaba que dijera el estado antes de mirarlo? | Espero que pare de decir running y cambie a failed, porque systemd ya no encuentra la ubicación real del archivo en el disco |
| ¿Qué dijo el estado? (código exacto) | failed (Result: exit-code) status=203/EXEC -> Result indica que fue un error causado dentro del servidor |
| ¿Qué dijo el registro? (frase textual) | tel223-latifo.service: Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory |
| ¿Cuál era la causa? | El llenado en el campo ExecStart de un archivo o ruta de archivo inexistente |
| ¿Cómo la corregí? | La corregí al volver a modificar el archivo de texto nano con la ruta de archivo correcta, guardarla, realizar un daemon-reload (reinicio del sistema) y volver a cargar el servicio con restart |
| ¿Cómo verifiqué que quedó corregida? | Al revisar en la respuesta del status lo siguiente: Active: active y no failed, además no hubo un result:exit-code (falla interna), tampoco un status=203/EXEC o error en ejecución y menos un registro que indique la causa de alguna falla. |

### Falla B — el permiso que falta

| Pregunta | Mi respuesta |
|---|---|
| ¿Qué esperaba que dijera el estado antes de mirarlo? | Espero que Active indique failed, ya que, el archivo .service se mantiene solo como de lectura o escritura y no se ejecuta. |
| ¿Qué dijo el estado? (código exacto) | Failed (Result: exit-code) -> Problema interno del servidor status=203/EXEC |
| ¿El código es el mismo que en la falla A? | Posee el mismo código del estado que la Falla A 203/EXEC -> falla en ejecución |
| ¿Qué dijo el registro? (frase textual) | tel223-latido.service:Failed at step EXEC spawning /usr/local/bin/latido.sh: Permission denied |
| ¿Cuál era la causa? | Quitar con chmode (change mode) el permiso a ejecución |
| ¿Cómo la corregí y cómo lo verifiqué? | Lo corregí al volver a añadir el permiso de ejecución del archivo tel223-latido.service con chmod (change mode) +x y volver a ejecutar el subproceso/servicio latido con restart |

Aún así, en el registro rojo se indica que systemd no pudo pasarle el archivo .service al programa execve porque no tenía permiso de ejecución: Permission denied, en la Falla A era: No such file or directory (podía ejecutarlo pero no lo encontraba)

**La conclusión**

| Pregunta | Mi respuesta |
|---|---|
| Dos causas distintas dieron el mismo código de error. ¿Qué me permitió distinguirlas? | Lo que permitió distinguirlas fue la lectura de los registros de cada falla (Permission denied o No such file or directory), indicando la causa. |
| Si mañana un servicio de mi proyecto no arranca, ¿cuál es el primer comando que ejecuto y por qué? | El primer comando que ejecuto es systemctl status nombre_del_servicio<br>para ver qué tipo de error es el que está pasando (puede ser interno Result: exit-code o externo Result:signal). |

| Falla / Pregunta | Respuesta requerida en Bitácora |
|---|---|
| **Falla A - Estado esperado** | Se esperaba un estado de falla ( `failed` )[cite: 1]. |
| **Falla A - Estado real (Código)** | `failed (Result: exit-code)` - `status=203/EXEC` [cite: 1] |
| **Falla A - Registro (Frase textual)** | `Failed at step EXEC spawning /usr/local/bin/latido-x.sh: No such file or directory` [cite: 1] |
| **Falla A - Causa y Corrección** | El archivo especificado en `ExecStart` no existía[cite: 1]. Se corrigió ajustando la ruta en el `.service` y ejecutando `daemon-reload` [cite: 1]. |
| **Falla B - Estado esperado** | Se esperaba un estado `failed` [cite: 1]. |
| **Falla B - Estado real (Código)** | `failed (Result: exit-code)` - `status=203/EXEC` [cite: 1] |
| **Falla B - ¿El código es el mismo?** | **Sí**, es exactamente el mismo código ( `203/EXEC` )[cite: 1]. |

| | |
|---|---|
| **Falla B - Registro (Frase textual)** | `Failed at step EXEC spawning /usr/local/bin/latido.sh: Permission denied` [cite: 1] |
| **Falla B - Causa y Corrección** | Falta de permiso de ejecución[cite: 1]. Se corrigió con `chmod +x /usr/local/bin/latido.sh` y reiniciando el servicio[cite: 1]. |
| **Conclusión - Distinción de fallas** | El comando `journalctl` (registro) permite ver el mensaje descriptivo específico del error, ya que el estado del sistema (`systemctl status`) solo muestra el código abstracto devuelto[cite: 1]. |
| **Conclusión - Primer comando futuro** | `journalctl -u nombre-servicio -n 20` para leer la causa real detallada de la falla en los registros[cite: 1]. |